

Compiler

Segment 2

Md. Nazmul Arefin

Lecturer

Dept. of Computer Science & Engineering
International Islamic University Chittagong

Syntax Definition

- A grammar naturally **describes** the **hierarchical structure** of most programming language constructs.

if(expression) statement ***else*** statement

- Using the variable `expr` to denote an expression and the variable `stmt` to denote a statement, this structuring rule can be expressed as :

`stmt -> if( expr ) stmt else stmt`

- Such a rule is called a **production**.
- In a production, lexical elements like the keyword ***if*** and the **parentheses** are called **terminals**.
- Variables like ***expr*** and ***stmt*** represent sequences of terminals and are called **nonterminals**.

Definition of Grammars

A *context-free-grammar* has four components:

1. A set of terminal symbols, sometimes referred to as "**tokens**." The terminals are the elementary symbols of the language defined by the grammar.
2. A set of nonterminals, sometimes called "**syntactic variables**." Each nonterminal represents a set of strings of terminals, in a manner we shall describe.
3. A set of productions, where each production consists of a nonterminal, called the head or left side of the production, an arrow, and a sequence of terminals and/or nonterminals, called the body or right side of the production.
4. A designation of one of the nonterminals as the start symbol

Definition of Grammars

Example:

9-5+2

Since a plus or minus sign must appear between two digits, we refer to such expressions as :

“lists of digits separated by plus or minus signs”

The following grammar describes the syntax of these expressions:

$list \rightarrow list + digit$	1
$list \rightarrow list - digit$	2
$list \rightarrow digit$	3
$digit \rightarrow 0 1 2 3 4 5 6 7 8 9$	4

Definition of Grammars

Example:

9-5+2

Since a plus or minus sign must appear between two digits, we refer to such expressions as :

“lists of digits separated by plus or minus signs”

The bodies of the three productions with nonterminal list as head equivalently can be grouped:

list -> *list* + *digit* | *list* - *digit* | *digit*

the terminals of the grammar are the symbols:

+ - 0 1 2 3 4 5 6 7 8 9

Grammars

- **Left linear Grammar:**

In a grammar if all productions are in the form $A \rightarrow B\alpha$ or $A \rightarrow \alpha$, where $A, B \in V_N$ and $\alpha \in \Sigma^*$ then the grammar is called left linear grammar.

e.g. $A \rightarrow Aa \mid Ba \mid b$

- **Right linear Grammar:**

In a grammar if all productions are in the form $A \rightarrow \alpha B$ or $A \rightarrow \alpha$, where $A, B \in V_N$ and $\alpha \in \Sigma^*$ then the grammar is called right linear grammar.

e.g. $A \rightarrow aA \mid aB \mid b$

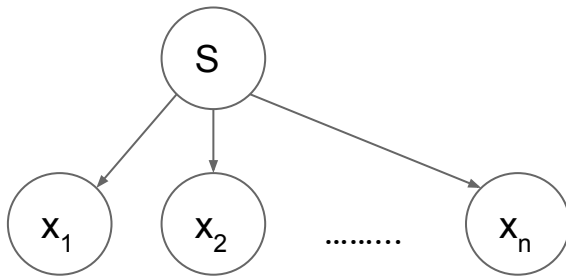
Derivations

- A grammar derives strings by beginning with the start symbol and repeatedly replacing a nonterminal by the body of a production for that nonterminal.
- The terminal strings that can be derived from the start symbol form the language defined by the grammar.
- we can deduce that $9-5+2$ is a list as follows.
 - a) 9 is a list by production (3), since 9 is a digit.
 - b) $9-5$ is a list by production (2), since 9 is a list and 5 is a digit.
 - c) $9-5+2$ is a list by production (1), since $9-5$ is a list and 2 is a digit.

$list \rightarrow list + digit \mid list - digit \mid digit$

Generation of Derivation Tree

- A derivation tree or parse tree is an ordered rooted tree that graphically represents the semantic information of a string derived from a context-free grammar.
- Properties of parse tree:
 - **Root** – Must be labeled by the start symbol.
 - **Interior nodes** – Labeled by a non-terminal symbol.
 - **Leaves** – Labeled by a terminal symbol or ϵ .
- If $S \rightarrow x_1, x_2, \dots, x_n$ is a production rule in a CFG, then the parse tree / derivation tree will be as follows –



Generation of Derivation Tree

There are two different approaches to draw a derivation tree –

Top-down Approach –

- Starts with the starting symbol S
- Goes down to tree leaves using productions

Bottom-up Approach –

- Starts from tree leaves
- Proceeds upward to the root which is the starting symbol S

Generation of Derivation Tree

- The derivation of $9-5+2$

$list \rightarrow list + digit \mid list - digit \mid digit$

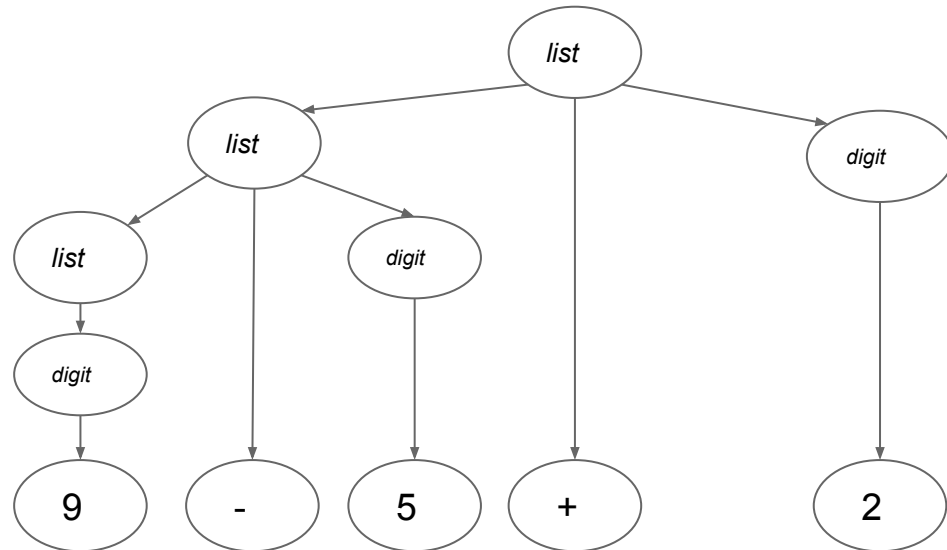
$List \rightarrow list + digit$

$\rightarrow list + digit + 2$

$\rightarrow list + 5 + 2$

$\rightarrow digit + 5 + 2$

$\rightarrow 9 + 5 + 2$



Generation of Derivation Tree

Example:

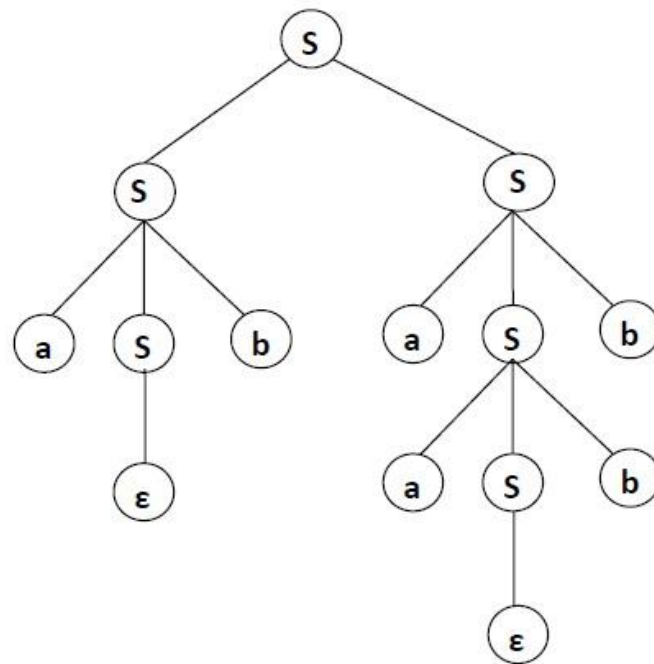
Let a CFG $\{N, T, P, S\}$ be

$N = \{S\}$, $T = \{a, b\}$, Starting symbol = S ,

$P = S \rightarrow SS \mid aSb \mid \epsilon$

One derivation from the above CFG is “abaabb”

$S \rightarrow SS \rightarrow aSbS \rightarrow abS \rightarrow abaSb \rightarrow abaaSbb \rightarrow abaabb$



Generation of Derivation Tree

Leftmost and Rightmost Derivation

- **Leftmost derivation** – A leftmost derivation is obtained by applying production to the leftmost variable in each step.
- **Rightmost derivation** – A rightmost derivation is obtained by applying production to the rightmost variable in each step.

Generation of Derivation Tree

Example:

Let any set of production rules in a CFG be

$$X \rightarrow X+X \mid X^*X \mid X \mid a$$

over an alphabet $\{a\}$.

The leftmost derivation for the string "a+a*a" may be –

$$X \rightarrow X+X \rightarrow a+X \rightarrow a + X^*X \rightarrow a+a^*X \rightarrow a+a^*a$$

Generation of Derivation Tree

Example:

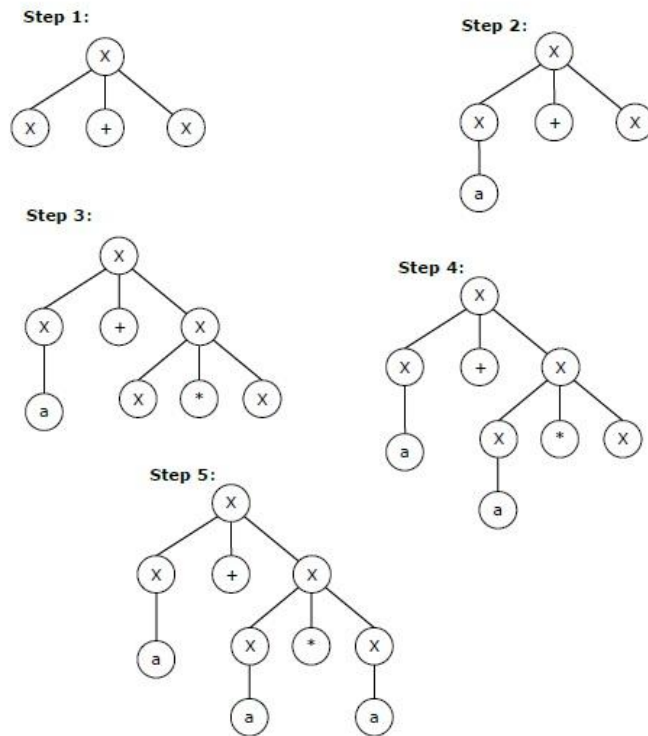
Let any set of production rules in a CFG be

$$X \rightarrow X+X \mid X^*X \mid X \mid a$$

over an alphabet $\{a\}$.

The leftmost derivation for the string "a+a*a" may be –

$$X \rightarrow X+X \rightarrow a+X \rightarrow a + X^*X \rightarrow a+a^*X \rightarrow a+a^*a$$



Generation of Derivation Tree

Practice:

1. Consider the grammar $\mathbf{p} \rightarrow \mathbf{aPbP} \mid \mathbf{bPaP} \mid \epsilon$
Show that *abab* can be generated by the grammar

Ambiguity in Context-Free Grammars

If a context free grammar G has more than one derivation tree for some string $w \in L(G)$, it is called an ambiguous grammar. There exist multiple right-most or leftmost derivations for some string generated from that grammar.

Problem

Check whether the grammar G with the following production rules is ambiguous or not.

$$X \rightarrow X+X \mid X*X \mid X \mid a$$

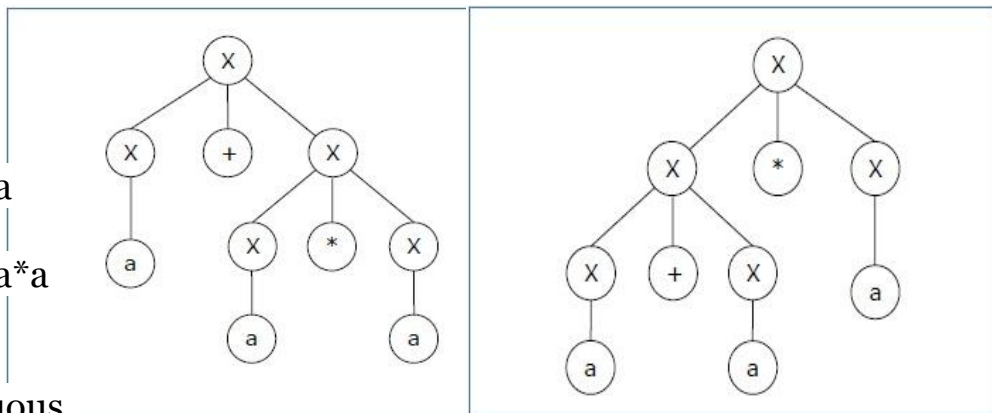
Derivation 1 –

$$X \rightarrow X+X \rightarrow a+X \rightarrow a+X*X \rightarrow a+a*X \rightarrow a+a*a$$

Derivation 2 –

$$X \rightarrow X*X \rightarrow X+X*X \rightarrow a+X*X \rightarrow a+a*X \rightarrow a+a*a$$

Since there are two parse trees for a single string "a+a*a", the grammar G is ambiguous.



Thank you